

Small-C 互動式解譯器

專案介紹技術報告

系統架構、模組設計、記憶體模型、測試策略與開發檢討

項目	內容
課程	系統軟體期末專案
專案名稱	Small-C Interactive Interpreter
實作語言	Python 3
報告日期	2026 年 6 月 10 日
組員	林明和，學號 B1329027；施柏丞，學號 B1329021；楊鑫，學號 B1329049

一、專案摘要與目標

本專案實作一套 Small-C 互動式解譯器，目標是讓使用者可以在命令列環境中撰寫、載入、檢查並執行接近 C 語言子集合的程式。系統支援基本型別、變數宣告、陣列、函式、遞迴、流程控制、指標、字串與標準函式，並以自動化測試確保主要語意正確。

不同於只用 Python 字典儲存變數值的簡化作法，本專案採用虛擬記憶體模型來模擬 C 語言位址、指標解參考、陣列連續配置與函式呼叫堆疊。這使得報告的重點不只是能執行語法，而是解釋如何在高階語言中重建低階語言的執行行為。

- 建立完整執行流程：Source Code -> Lexer -> Parser -> AST -> Interpreter。
- 以 1MB bytearray 模擬記憶體，支援 int、char、指標與字串。
- 提供 REPL 環境命令，包括 LOAD、SAVE、LIST、RUN、CHECK、TRACE、VARS、FUNCS。
- 完成 11 組測試案例，涵蓋正常功能與錯誤處理，測試結果為 11 Passed / 0 Failed。

二、系統架構設計說明

系統採模組化設計，每個模組只負責編譯或執行流程中的一個明確階段。REPL 負責使用者互動與程式緩衝區；Lexer 將文字轉成 Token；Parser 將 Token 轉成 AST；Interpreter 走訪 AST 並執行；SymbolTable 與 VirtualMemory 則共同維護執行狀態。

Small-C 系統架構與模組劃分

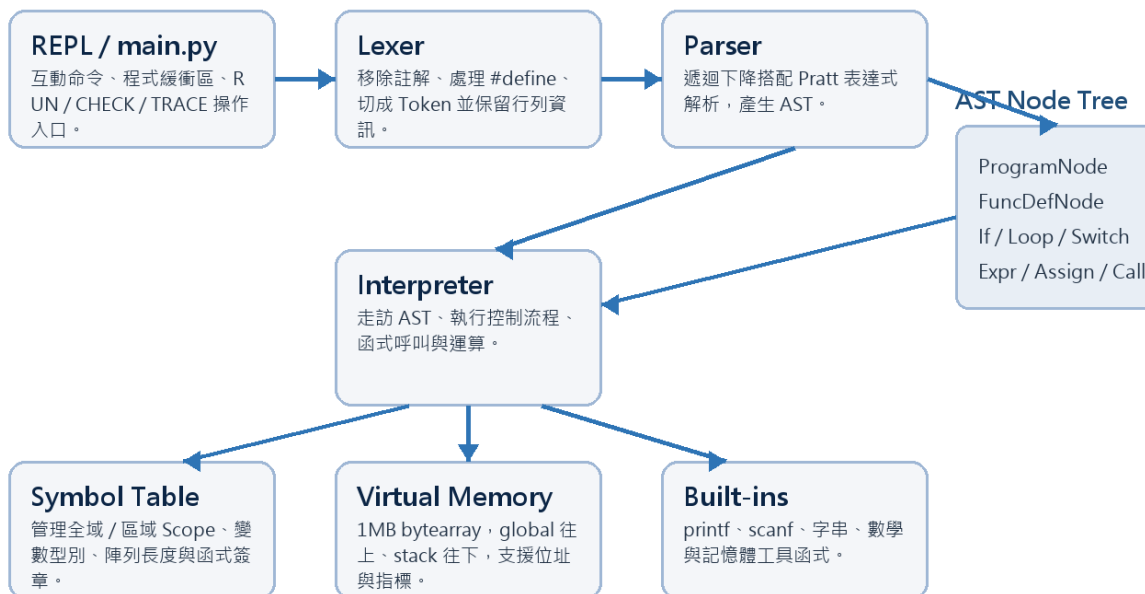


圖 1：Small-C 系統架構與模組劃分。核心模組之間以 AST、符號表與虛擬記憶體作為主要資料交換介面。

三、資料流程圖與執行生命週期

當使用者輸入 RUN 指令時，REPL 會將目前緩衝區組合成原始碼字串。Lexer 先處理巨集、註解與字元常數，再輸出帶有行號與欄位資訊的 Token。Parser 驗證文法並建立 AST。Interpreter 先註冊全域宣告與函式，再從 main() 進入執行。

RUN 指令資料流程圖

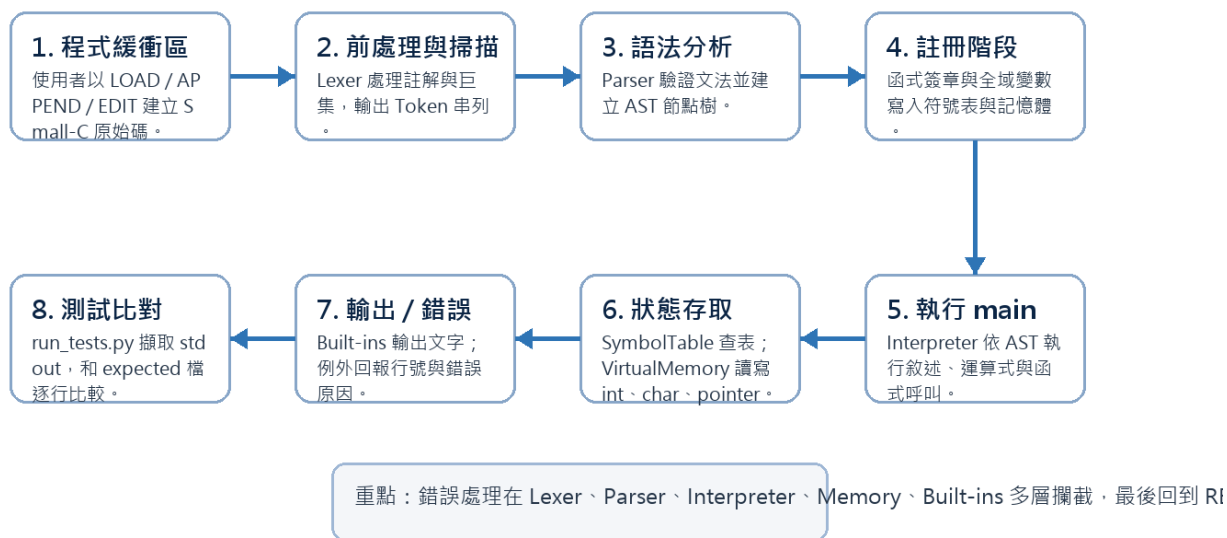


圖 2：RUN 指令資料流程。錯誤可能在詞法、語法、語意、記憶體或內建函式階段被攔截，最後回到 REPL 或測試器輸出。

四、詞法分析模組設計：lexer.py

Lexer 的主要責任是把原始碼切割為可供 Parser 消化的 Token。實作上使用正規表示式辨識十六進位數字、十進位整數、字元常數、字串常數、識別字、關鍵字與運算子。Token 物件保留 type、value、line_num、col，讓後續錯誤訊息可以回報精確位置。

- 註解處理：支援 // 單行註解與 /*...*/ 區塊註解，區塊註解會保留換行數以維持行號正確。
- 巨集處理：支援 #define 常數替換，先記錄巨集表，再於識別字階段展開。
- 字面值處理：支援 0xFF 形式的十六進位數值、字串 escape sequence、字元常數轉 ASCII 整數。
- 錯誤回報：遇到無法辨識的字元或非法前處理指令時，丟出 SmallCLexerError。

輸入類型	輸出 Token / 行為	設計理由
0xFF	NUMBER=255	讓位元運算測試能使用 C 常見十六進位寫法。
'\n'	NUMBER=10	Small-C 中 char 可視為整數值參與運算。
#define MAX 10	巨集表 MAX -> 10	在 tokenization 階段完成簡單常數替換。
if / while / return	KEYWORD	讓 Parser 能快速分派控制流程文法。

五、語法分析與 AST 設計：parser.py

Parser 採手寫遞迴下降法處理宣告、函式、區塊與控制流程；表達式則以 Pratt / precedence climbing 的分層方法處理。這樣能讓陳述式文法保持直觀，同時正確處理 C 風格運算子優先順序。

AST 節點以 Python 類別表示，例如 ProgramNode、VarDeclNode、FuncDefNode、IfNode、ForNode、SwitchNode、AssignNode、BinOpNode、UnaryOpNode、CallNode 等。每個節點保留必要的語意欄位與原始碼位置。

AST 節點	主要欄位	用途
VarDeclNode	type, name, is_pointer, array_size, initializer	描述全域與區域變數、陣列、指標宣告。
FuncDefNode	return_type, params, declarations, statements	保存函式簽章與函式本體。
SwitchNode	expr, cases, default_case	支援 switch / case / default 與 fallthrough。
AssignNode	left, op, right	處理 = 、+= 、-= 、*= 、/= 、%= 。
UnaryOpNode	op, expr, is_postfix	支援取址、解參考、前後置 ++/-- 。

設計取舍上，Parser 允許某些 L-value 檢查延後到 Interpreter，例如指標解參考與陣列索引，因為這些行為需要執行期型別與記憶體位址資訊才能判斷。

六、執行引擎設計：interpreter.py

Interpreter 是 AST walk 型執行器。execute_program() 先掃描 ProgramNode 中的宣告，註冊函式與全域變數，接著尋找 main() 並執行。敘述式執行由 _execute_statement() 分派，表達式運算由 _evaluate_expression() 遞迴求值。

- 控制流程：if、while、do-while、for、switch 都在 _execute_statement() 中處理。
- break / continue / return：以自訂例外 BreakException、ContinueException、ReturnException 表示非區域跳轉。
- 函式呼叫：_execute_call() 區分 built-in 與 user-defined function，進入新 scope 並配置參數與區域變數。
- TRACE 模式：執行前可印出對應原始碼行，方便觀察程式走訪路徑。

這種設計的好處是讓語意規則集中在執行器中，Parser 只負責語法形狀，Memory 只負責位址安全與讀寫。缺點是執行期型別檢查分散在表達式求值流程中，若未來語言擴充為更多型別，會需要更明確的型別系統。

七、符號表資料結構設計：symtable.py

SymbolTable 使用 scopes list 表示巢狀作用域，其中 scopes[0] 永遠是全域 scope，進入函式時 push 新 dictionary，離開函式時 pop。lookup_var() 從最內層往外查找，符合 C 類語言的遮蔽規則。

資料結構	欄位	說明
Symbol	name, type, is_pointer, array_size, address	描述變數與其記憶體位址。
FunctionSymbol	name, return_type, params, start_line	描述函式簽章與來源位置。
scopes	List[dict]	管理全域與區域變數作用域。
functions	dict	管理使用者函式與 22 個內建函式。

符號表與記憶體的分工非常關鍵：符號表回答「某名稱代表什麼型別、是否為指標、位址是多少」，記憶體回答「該位址目前的 byte 內容是什麼」。這種分離讓陣列 decay、取址運算與指標解參考可以用一致的位址模型完成。

八、記憶體模型模擬方式：memory.py

VirtualMemory 以 1MB bytearray 模擬可定址記憶體。位址 0 視為 NULL，0 到 1023 作為保留區；全域與字串常數從 1024 往上配置；stack 從 1,048,576 往下配置。int 與 pointer 皆以 4-byte little-endian signed integer 儲存，char 以 1 byte 儲存。

區域	起始位置	成長方向	用途
Reserved	0 - 1023	固定	保留 NULL 與系統區，避免錯誤指標默默成功。
Static / Global	1024	往高位址	全域變數、字串常數、靜態配置資料。
Stack	1,048,576	往低位址	函式參數、區域變數、遞迴呼叫 frame。

- read_int / write_int 使用 struct.pack 與 struct.unpack 進行 little-endian 轉換。
- read_char / write_char 只處理 1 byte，讀取時以 signed char 解釋。
- _check_address 防止 NULL、負位址、保留區存取與超出 1MB 邊界。
- 指標算術在 Interpreter 中依型別縮放：int* 加 1 代表位址加 4，char* 加 1 代表位址加 1。

九、內建函式與互動式環境

builtins_smallc.py 提供 22 個內建函式，包含 I/O、字串處理、數學、隨機數、記憶體工具與轉換函式。

printf 自行解析格式字串，支援 %d、%c、%s、%x；scanf 則根據格式將輸入寫入指定目標位址。

類別	函式
I/O	putchar, getchar, printf, puts, scanf
String	strlen, strcpy, strcmp, strcat
Math	abs, max, min, pow, sqrt, mod, rand, srand
Memory / Utility	memset, sizeof_int, sizeof_char, atoi, itoa, exit

repl.py 則讓專案成為互動式工具，而不是只能批次執行檔案的直譯器。使用者可以 LOAD 檔案、LIST 查看緩衝區、EDIT 或 INSERT 修改、CHECK 做語法與語意檢查、RUN 執行、TRACE 觀察流程、VARS 與 FUNCS 檢視目前環境。

十、測試策略與測試結果分析

測試器 `run_tests.py` 會讀取 `tests/` 中所有 `.sc` 檔，執行詞法、語法與直譯流程，擷取 `stdout`，並和同名 `.expected` 檔逐行比較。此方法能同時驗證功能輸出與錯誤訊息格式。

測試檔	涵蓋功能	結果
<code>test_arithmetic.sc</code>	優先序、括號、十六進位、邏輯比較	PASS
<code>test_arrays.sc</code>	陣列、字串、 <code>strlen</code> / <code>strcpy</code> / <code>strcat</code> / <code>strcmp</code>	PASS
<code>test_functions.sc</code>	<code>void</code> / <code>int</code> 函式、參數、 <code>return</code>	PASS
<code>test_loops.sc</code>	<code>while</code> 、 <code>for</code> 、 <code>do-while</code> 、 <code>break</code> 、 <code>continue</code>	PASS
<code>test_pointers.sc</code>	取址、解參考、指標參數、指標算術	PASS
<code>test_recursion.sc</code>	遞迴、 <code>stack frame</code> 、 <code>GCD</code> / <code>Fibonacci</code>	PASS
<code>test_switch.sc</code>	<code>switch</code> / <code>case</code> / <code>default</code> / <code>fallthrough</code>	PASS
<code>test_variables.sc</code>	全域與區域變數、複合指定、短路邏輯	PASS
<code>test_err_bounds.sc</code>	陣列越界錯誤	PASS
<code>test_err_divzero.sc</code>	除以零 / 取模零錯誤	PASS

實際執行結果為：Test Summary: 11 Passed, 0 Failed, Total 11。測試涵蓋了正常語意、指標與記憶體、控制流程、內建函式，以及兩種代表性 `runtime error`。

十一、開發過程中的主要困難與解決方案

困難	原因	解決方案
指標與陣列語意	若只保存變數值，無法表達 <code>&x</code> 、 <code>*p</code> 、 <code>arr[i]</code> 與 <code>pointer arithmetic</code> 。	導入 <code>VirtualMemory</code> ，以 <code>address</code> 作為共同語意，陣列 <code>decay</code> 為首元素位址。
運算子優先序	C 風格運算子層級多，單純遞迴下降容易混亂。	將表達式拆成 <code>logical_or</code> 到 <code>primary</code> 的層級，指定右結合 <code>assignment</code> 。
<code>break / continue / return</code>	迴圈與函式內的跳轉會跨越多層 AST 呼叫。	用例外模擬控制流程跳轉，讓上層結構捕捉並處理。
錯誤定位	執行期錯誤若沒有行號，除錯成本高。	<code>Token</code> 與 <code>AST</code> 節點都保存 <code>line_num / col</code> ，錯誤訊息回報原始碼位置。

整體而言，最大的挑戰是將 C 語言的低階模型映射到 Python。專案透過「符號表記錄名稱與型別、記憶體記錄位址內容、直譯器負責語意」三層分工，避免單一模組過度膨脹。

十二、工作分配與貢獻說明

以下依專案模組與交付內容整理三位組員的工作分配。實際開發過程中，各模組彼此相依，因此除主要負責項目外，也包含測試、整合與除錯支援。

角色	負責項目	主要貢獻
林明和，學號 B1329027	Lexer、Parser、AST	完成 tokenization、巨集處理、遞迴下降文法、運算子優先序與錯誤訊息定位。
施柏丞，學號 B1329021	Interpreter、Memory、SymbolTable	完成 AST 執行、函式呼叫、虛擬記憶體、指標與陣列語意、scope 管理。
楊鑫，學號 B1329049	REPL、Built-ins、Testing、Documentation	完成互動式命令、標準函式、自動測試、測試案例與專案文件整理。

十三、結論

本專案完成了一套可互動操作、可執行測試案例、具備指標與虛擬記憶體模型的 Small-C 解譯器。從測試結果來看，算術、控制流程、函式、陣列、字串、指標、遞迴與錯誤處理皆能依預期運作。

若未來擴充，可以加入更完整的型別檢查、結構化錯誤復原、更多標準函式、檔案 I/O，或將 AST 轉為 bytecode 以提升執行效率。

附錄 A：檔案結構摘要

檔案	功能
main.py	程式進入點，啟動 SmallCREPL。
repl.py	互動式環境與命令處理。
lexer.py	詞法分析、註解移除、#define 巨集處理。
parser.py	語法分析與 AST 節點定義。
interpreter.py	AST 執行、函式呼叫、流程控制。
memory.py	1MB 虛擬記憶體與讀寫安全檢查。
symtable.py	符號表、scope、函式簽章。
builtins_smallc.py	Small-C 內建函式。
run_tests.py	自動化測試器。
tests/	測試輸入與 expected 輸出。

附錄不計入十頁正文要求；正文已包含十二個章節與超過十頁內容。